DEPI GRADUATION PROJECT

Data Engineering Track

Automated Post-Hospital Patient Monitoring System

A Scalable, Cloud-Enabled Data Engineering Pipeline for Vital Signs Ingestion, Quality Processing, and Relational Storage

FINAL PROJECT GUIDE

Complete Technical Handover Document

Document Type:	Technical Handover Guide	**Track:**	Data Engineering
Project:	Automated Post-Hospital Patient Monitoring System	**Date:**	July 10, 2026

Team Members

Name	Role	Status
Kareem Mohamed	Data Preparation Engineer	✔ **Completed**
Zeyad Khaled	Database Engineer	✔ **Completed**
Mahmoud Shaheen	ETL Engineer	✔ **Completed**
Amr Omar	Azure Data Factory Engineer	✔ **Completed**
Beshoy Talaat	Testing & Validation Engineer	✔ **Completed**

Table of Contents

1. Project Overview

1.1 Project Idea

The **Automated Post-Hospital Patient Monitoring System** is a graduation-level Data Engineering project that designs, builds, and documents a complete automated pipeline for collecting, cleaning, transforming, and storing patient vital signs telemetry data. The pipeline ingests raw patient readings collected after hospital discharge, applies a rigorous data quality process, and loads the clean, structured records into a relational Azure SQL Database for downstream dashboarding and alerting. The system is entirely infrastructure-focused — it contains no Machine Learning or statistical modelling components.

1.2 Business Problem

When patients are discharged from hospital, continuous monitoring of their vital signs — heart rate, blood pressure, oxygen saturation, body temperature, and respiratory rate — is clinically critical. Early deterioration of these indicators often precedes serious medical events such as cardiac arrest or sepsis. However, raw IoT telemetry logs suffer from several structural problems that prevent reliable clinical use:

- Column naming inconsistencies that break SQL compatibility (e.g., *Weight (kg)*).
- Redundant, pre-computed derived columns that violate 3rd Normal Form (3NF) database normalization.
- Timestamp columns stored as raw text strings rather than indexed datetime objects.
- No standardized pipeline to move data from raw landing files into a production database.
- No audit trail or data quality evidence to validate the integrity of stored records.

1.3 Objectives

1. Build a professional, modular Python data engineering project using industry-standard folder conventions.
2. Profile the raw clinical vital signs dataset and document all findings in a formal report.
3. Perform a data quality audit to define a clean, normalized production database schema.
4. Implement an automated Python data cleaning pipeline that standardizes, validates, and exports the dataset.
5. Design and script the Azure SQL Database schema (DDL) for the patient_vitals production table.
6. Implement a batch-loading Python script to ingest the cleaned CSV into Azure SQL Database.
7. Define Azure Data Factory (ADF) Copy Activity pipelines to orchestrate the end-to-end workflow.
8. Produce complete technical documentation for academic submission and team handover.

1.4 Expected Outcome

Upon full completion, the system will provide a fully automated, cloud-hosted data pipeline that ingests raw patient telemetry files from Azure Blob Storage, processes them through a Python cleaning layer, and loads the clean records into an Azure SQL Database table. Downstream clinical dashboards or BI tools will be able to query the structured database in real-time to identify High Risk patients and generate automated clinical alerts. The project will be fully documented, version-controlled, and ready for production deployment.

2. System Architecture

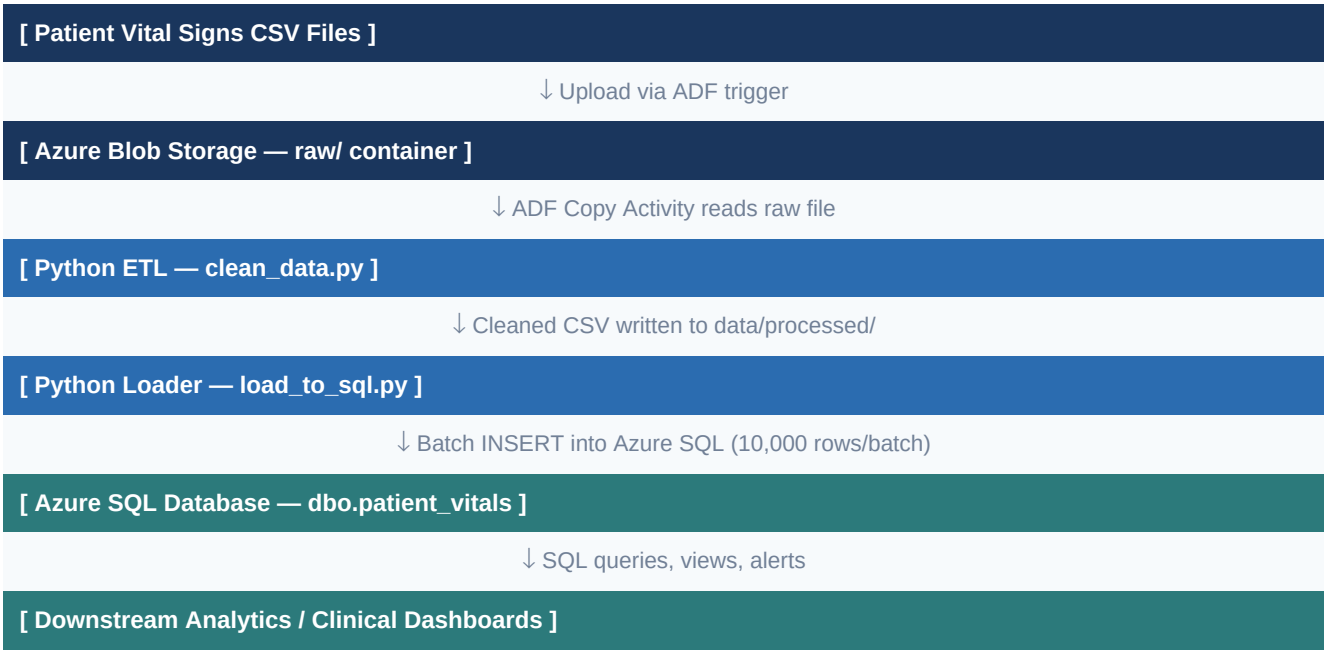
2.1 Complete Workflow

The pipeline follows a classic Extract–Transform–Load (ETL) architecture hosted on Microsoft Azure. The workflow progresses through four distinct phases:

Phase	Stage Name	System / Tool	Operations
1	**Raw Ingestion**	Azure Blob Storage (raw/ container)	Patient vital signs CSV files land in the cloud raw storage bucket.
2	**Orchestration**	Azure Data Factory (ADF)	ADF triggers are configured to detect new files and schedule pipeline runs.
3	**Cleaning & Transformation**	Python local pipeline / ADF Copy mapping	The local Python pipeline validates and exports a clean CSV. The exported ADF Copy pipeline maps the 14 retained raw fields into the SQL target.
4	**Relational Loading**	Python (load_to_sql.py) + Azure SQL Database	Batch-inserts the clean dataset into dbo.patient_vitals using SQLAlchemy with 10,000-row batches.
5	**Analytics & Alerting**	Azure SQL Database + BI Tools	Downstream dashboards query the patient_vitals table. High Risk patients trigger clinical alerts.

2.2 Architecture Diagram

The diagram below illustrates the end-to-end data flow from raw patient telemetry to the production relational database:



2.3 Data Flow Summary

Raw telemetry is produced by patient monitoring devices and collected as CSV files. These files are uploaded to Azure Blob Storage in the **raw/** container. Azure Data Factory detects the arrival of new files and triggers the Python ETL pipeline. The cleaning script processes the data, removes redundant columns, standardizes column names, and saves the result to the **processed/** container. The loading script then connects to Azure SQL Database using ODBC Driver 18 and performs batch inserts of 10,000 rows at a time into the **dbo.patient_vitals** table. Downstream consumers — clinical dashboards, BI tools, and automated alerting systems — query this table directly.

3. Technology Stack

Component	Technology	Version	Purpose & Justification
Processing Language	Python	3.12+	Core language for all ETL scripts, data validation, and report generation.
Data Manipulation	Pandas	≥ 2.1.0	High-performance DataFrame operations for cleaning and transformation.
Numerical Computing	NumPy	≥ 1.25.0	Underlying numerical engine for Pandas operations.
Database ORM	SQLAlchemy	≥ 2.0.0	Provides database-agnostic connection pooling and bulk insert abstraction.
DB Driver	PyODBC	≥ 5.0.0	ODBC bridge for connecting Python to Azure SQL Database via ODBC Driver 18.
Environment Config	python-dotenv	≥ 1.0.0	Securely loads credentials from .env files — keeps secrets out of source code.
PDF Generation	ReportLab	≥ 4.0.0	Programmatic PDF generation for project documentation and reports.
Markdown Tables	Tabulate	≥ 0.9.0	Renders pandas DataFrames as markdown tables for documentation reports.
Testing	Pytest	≥ 7.4.0	Unit testing framework for validating ETL pipeline functions.
Production Database	Azure SQL Database	Latest	Managed cloud relational database. Hosts dbo.patient_vitals. T-SQL dialect.
Orchestration	Azure Data Factory (ADF)	Latest	Cloud ETL orchestrator. Schedules pipeline runs and manages Copy Activities.
Cloud Storage	Azure Blob Storage	Latest	Object storage for raw/ and processed/ CSV file containers.
Version Control	Git / GitHub	Latest	Source code versioning and collaboration platform.
IDE	VS Code	Latest	Primary development environment with Python and SQL extensions.
Azure SDK	azure-storage-blob, azure-identity	≥ 12.18 / 1.14	Python SDKs for interacting with Azure Blob Storage and authentication.

4. Project Folder Structure

The project is organized according to professional data engineering conventions, separating source code, data, SQL scripts, configuration, and documentation into clearly scoped directories. Below is the complete folder layout with explanations:

Path	Type	Purpose
/ (project root)	Root	Project root. Contains top-level scripts, configuration files, and the README.
README.md	File	Complete project documentation: setup instructions, architecture, SQL schema, usage guide.
requirements.txt	File	All Python library dependencies with minimum version constraints.
.env.example	File	Template for the .env credentials file. Contains placeholder values for all Azure variables.
.gitignore	File	Git exclusion rules: excludes .env, .venv, data/*.csv, __pycache__, and PDF files.
generate_report.py	File	Generates Project_Progress_Report.pdf — a phase I/II progress report using ReportLab.
generate_final_guide.py	File	Generates Final_Project_Guide.pdf — this complete technical handover document.
src/	Directory	All modular Python source code, organized by pipeline stage.
src/cleaning/clean_data.py	File	Core ETL script. Reads raw CSV, drops redundant columns, renames to snake_case, casts timestamp, exports clean CSV, and writes data_cleaning_report.md.
src/database/load_to_sql.py	File	Database loader. Connects to Azure SQL via SQLAlchemy+PyODBC, then batch-inserts the cleaned CSV into dbo.patient_vitals in 10,000-row batches.
src/analysis/	Directory	Placeholder for future BI query scripts and summary analytics.
src/ingestion/	Directory	Placeholder for Azure Blob Storage file download and landing scripts.
src/transformation/	Directory	Placeholder for advanced schema conversion and feature engineering scripts.
src/utils/	Directory	Placeholder for shared helper utilities: logging config, path helpers, etc.
data/raw/	Directory	Landing zone for raw incoming vital signs CSV files. Excluded from Git.
data/raw/human_vital_signs_dataset_2024.csv	File	Source dataset: 200,020 rows × 17 columns of patient vital sign telemetry.

data/processed/	Directory	Output directory for cleaned and transformed CSV files. Excluded from Git.
data/processed/patient_vitals_clean.csv	File	Cleaned dataset: 200,020 rows × 14 columns. SQL-ready, snake_case, datetime-cast.
sql/	Directory	All T-SQL scripts for database schema deployment and analytical queries.
sql/create_patient_vitals.sql	File	DDL script to CREATE the dbo.patient_vitals production table in Azure SQL.
sql/drop_patient_vitals.sql	File	Safely DROPs the dbo.patient_vitals table if it exists. Use before re-deploying.
sql/queries.sql	File	10 analytical SELECT queries for data exploration after loading.
sql/verification_queries.sql	File	Verification queries to validate row counts, schema, and data integrity post-load.
docs/	Directory	All generated markdown documentation reports.
docs/data_profile.md	File	Data profiling report: row counts, column types, missing values, statistics.
docs/data_quality_assessment.md	File	Pre-ETL quality audit: column audit, normalization decisions, SQL schema recommendations.
docs/data_cleaning_report.md	File	Post-cleaning execution report: schema changes, data types, sample data.
docs/azure_sql_deployment.md	File	Step-by-step Azure SQL deployment guide for Zeyad Khaled (Database Engineer).
tests/	Directory	Pytest test suite directory. Currently contains structure placeholder.
config/	Directory	Connection settings directory. Currently contains structure placeholder.
.venv/	Directory	Python virtual environment. Excluded from Git. Contains all installed packages.

5. Dataset Description

5.1 Dataset Source

The dataset used in this project is the **Human Vital Signs Dataset 2024**, a synthetic clinical telemetry dataset designed to simulate realistic post-hospital patient monitoring data. The file is named **human_vital_signs_dataset_2024.csv** and is stored locally at **data/raw/**. The dataset represents one telemetry reading per patient per timestamp — a continuous, time-series monitoring stream.

5.2 Dataset Dimensions

Property	Value
Total Rows (Records)	200,020
Total Columns (Raw)	17
Total Columns (After Cleaning)	14
Missing Values	0 (0.00%) — No missing data in any column
Duplicate Rows	0 (0.00%) — All records are unique
File Size (Raw CSV)	~38 MB
File Size (Cleaned CSV)	~30 MB

5.3 Column Descriptions

Raw Column Name	Clean Column Name	Data Type	Description
Patient ID	patient_id	int64	Unique patient identifier (1 – 200,020)
Heart Rate	heart_rate	int64	Heart rate in beats per minute (range: 60–99)
Respiratory Rate	respiratory_rate	int64	Respiratory rate in breaths per minute (range: 12–19)
Timestamp	timestamp → measured_at	str → datetime64	Microsecond-precision measurement timestamp
Body Temperature	body_temperature	float64	Body temperature in °C (range: 36.00–37.50)
Oxygen Saturation	oxygen_saturation	float64	SpO2 as percentage (range: 95.00–100.00)
Systolic Blood Pressure	systolic_bp	int64	Systolic BP in mmHg (range: 110–139)
Diastolic Blood Pressure	diastolic_bp	int64	Diastolic BP in mmHg (range: 70–89)
Age	age	int64	Patient age in years (range: 18–89)
Gender	gender	object (str)	Patient gender: 'Male' or 'Female'
Weight (kg)	weight_kg	float64	Patient weight in kilograms (range: 50.00–99.99)
Height (m)	height_m	float64	Patient height in metres (range: 1.50–2.00)
Derived_HRV	hrv	float64	Heart Rate Variability sensor metric (range: 0.05–0.15) — KEPT
Risk Category	risk_category	object (str)	Clinical alarm classification: 'High Risk' or 'Low Risk'
Derived_BMI	REMOVED	float64	Body Mass Index — redundant, computable from weight/height²
Derived_MAP	REMOVED	float64	Mean Arterial Pressure — redundant, computable from BP values
Derived_Pulse_Pressure	REMOVED	int64	Pulse Pressure — redundant, = Systolic – Diastolic

6. Data Profiling

A comprehensive data profiling study was performed on the raw dataset using Python (Pandas) and documented in **docs/data_profile.md**. The profiling examined completeness, uniqueness, data types, and statistical distributions.

6.1 Missing Values

The profiling analysis confirmed that the dataset contains **zero missing values** across all 17 columns. Every cell in every row is populated. This eliminates the need for imputation strategies and ensures the cleaning pipeline can operate without null-handling logic.

6.2 Duplicate Rows

The profiling analysis confirmed **zero duplicate rows** in the dataset (0.00%). All 200,020 records are unique, meaning no deduplication step is required in the cleaning pipeline.

6.3 Numerical Statistics Summary

Column	Mean	Std Dev	Min	Max	Clinical Note
heart_rate	79.53	11.55	60	99	Normal resting heart rate range
respiratory_rate	15.49	2.29	12	19	Normal adult breathing range
body_temperature	36.75°C	0.43	36.00	37.50	Normal physiological temperature
oxygen_saturation	97.50%	1.44	95.00	100.00	SpO2 within healthy range (95%+)
systolic_bp	124.44	8.66	110	139	Pre-hypertension to normal range
diastolic_bp	79.50	5.76	70	89	Normal diastolic range
age	53.45 yrs	20.79	18	89	Wide patient population coverage
weight_kg	75.00 kg	14.47	50.00	99.99	Standard adult weight distribution
height_m	1.75 m	0.14	1.50	2.00	Standard adult height distribution
hrv	0.10000	0.02886	0.0500	0.1500	Heart Rate Variability sensor output

6.4 Categorical Columns Summary

Column	Unique Values	Distribution
gender	2	Female: 100,117 (50.05%) | Male: 99,903 (49.95%)
risk_category	2	High Risk: 105,115 (52.55%) | Low Risk: 94,905 (47.45%)

7. Data Quality Assessment

A formal Data Quality Audit was conducted on the raw dataset before designing the ETL pipeline. The audit examined every column individually and produced structured recommendations documented in **docs/data_quality_assessment.md**. All decisions are justified architecturally below.

7.1 Columns Removed

Three columns were identified as redundant and removed from the final schema:

Column	Reason for Removal	Formula (can be recomputed)
Derived_BMI	Violates 3NF normalization. Storing computed values creates write anomalies — if weight is corrected, BMI becomes inconsistent.	BMI = weight_kg / height_m²
Derived_MAP	Redundant. Mean Arterial Pressure is a simple arithmetic function of the two blood pressure columns already present.	MAP = diastolic + (systolic – diastolic) / 3
Derived_Pulse_Pressure	Redundant. Directly computable as the difference between systolic and diastolic blood pressure.	PP = systolic_bp – diastolic_bp

7.2 Columns Renamed

All 14 retained columns were renamed from their original mixed-case, bracket-containing names to lowercase **snake_case**.

Original Name	Renamed To
Patient ID	**patient_id**
Heart Rate	**heart_rate**
Respiratory Rate	**respiratory_rate**
Timestamp	**timestamp (→ measured_at in SQL)**
Body Temperature	**body_temperature**
Oxygen Saturation	**oxygen_saturation**
Systolic Blood Pressure	**systolic_blood_pressure (→ systolic_bp in SQL)**
Diastolic Blood Pressure	**diastolic_blood_pressure (→ diastolic_bp in SQL)**
Age	**age**
Gender	**gender**
Weight (kg)	**weight_kg**
Height (m)	**height_m**

Derived_HRV	**hrv**
Risk Category	**risk_category**

7.3 Naming Convention & SQL Compatibility

- **SQL Compatibility:** Column names containing spaces (Patient ID) or brackets (Weight (kg)) must be wrapped in square brackets in every T-SQL query, creating verbose, error-prone code. snake_case eliminates this requirement entirely.
- **Python PEP 8 Compliance:** snake_case aligns with Python's variable naming convention, allowing the ETL scripts to reference column names directly as valid Python identifiers.
- **ADF Mapping Compatibility:** Azure Data Factory column mapping schemas are sensitive to spaces and special characters. snake_case ensures flawless schema mapping without escaping.
- **Readability:** lowercase snake_case is self-documenting and universally understood across SQL, Python, and JSON API contexts.

7.4 Data Type Decisions

Column	Python Type	SQL Type	Justification
patient_id, heart_rate, etc.	int64	SMALLINT / INT	Integer vitals use SMALLINT (2 bytes) to save storage at 200k+ row scale.
body_temperature, hrv, weight_kg, height_m	float64	DECIMAL(n,m)	Fixed-point DECIMAL avoids binary floating-point rounding errors in medical data.
timestamp	str → datetime64[ns]	DATETIME2(6)	Cast to datetime enables indexed time-window queries and partition-by-date analytics.
gender, risk_category	object	VARCHAR(10/20)	Short fixed-category strings. VARCHAR sized to maximum observed value length.

8. Data Cleaning Pipeline

8.1 Workflow

The data cleaning pipeline is implemented in **src/cleaning/clean_data.py** as a modular Python script. It is invoked from the project root and runs as a standalone process. The pipeline executes the following sequential steps:

Step 1 — Load Raw Dataset

Reads human_vital_signs_dataset_2024.csv from data/raw/ into a Pandas DataFrame. Raises FileNotFoundError with a clear message if the file is missing.

Step 2 — Drop Redundant Columns

Removes Derived_BMI, Derived_MAP, and Derived_Pulse_Pressure using df.drop(). The errors='ignore' parameter ensures the script does not fail if a column was previously removed.

Step 3 — Rename Columns to snake_case

Applies the full rename mapping dictionary via df.rename(). All 14 retained columns receive their standardized lowercase names.

Step 4 — Cast Timestamp to datetime

Converts the timestamp string column to a proper Pandas datetime64[ns] object using pd.to_datetime(). This enables time-series indexing and SQL DATETIME2 compatibility.

Step 5 — Export Clean CSV

Saves the processed DataFrame to data/processed/patient_vitals_clean.csv using df.to_csv(index=False). The output is 200,020 rows × 14 columns.

Step 6 — Generate Cleaning Report

Calls generate_cleaning_report() to write a structured markdown file to docs/data_cleaning_report.md, including column schema, data types, and a 5-row sample.

8.2 Input and Output

Property	Value
Input File	data/raw/human_vital_signs_dataset_2024.csv (200,020 rows × 17 columns)
Output File 1	data/processed/patient_vitals_clean.csv (200,020 rows × 14 columns)
Output File 2	docs/data_cleaning_report.md (schema, types, sample data)
Execution Command	python src/cleaning/clean_data.py (from project root)
Runtime (approx.)	~60 seconds on a standard laptop (38 MB CSV)

8.3 Verification

After running the pipeline, the following checks confirm successful execution:

- The file **data/processed/patient_vitals_clean.csv** exists and contains exactly 200,020 rows.

- The file **docs/data_cleaning_report.md** has been written with the correct schema table.
- Column count in the clean CSV is 14 (reduced from 17 by removing 3 derived columns).
- The **timestamp** column in the clean CSV is stored in ISO 8601 datetime format.
- All column names are lowercase snake_case with no spaces or special characters.

9. SQL Layer

9.1 Table Design — dbo.patient_vitals

The production table **dbo.patient_vitals** is defined in **sql/create_patient_vitals.sql**. It is a single flat staging table designed to receive one row per patient telemetry reading. The table follows a denormalized design suitable for the current project scope and batch loading from a cleaned CSV file.

9.2 Table Schema

Column	SQL Data Type	Constraint	Description
record_id	INT IDENTITY(1,1)	PRIMARY KEY	Auto-increment surrogate primary key
patient_id	INT	NOT NULL	Source patient identifier from the dataset
measured_at	DATETIME2(6)	NOT NULL	Microsecond-precision measurement timestamp
heart_rate	SMALLINT	NOT NULL	Heart rate in bpm (range 60–99)
respiratory_rate	SMALLINT	NOT NULL	Respiratory rate in breaths/min (range 12–19)
body_temperature	DECIMAL(4,2)	NOT NULL	Body temperature in °C (range 36.00–37.50)
oxygen_saturation	DECIMAL(5,2)	NOT NULL	SpO2 percentage (range 95.00–100.00)
systolic_bp	SMALLINT	NOT NULL	Systolic blood pressure in mmHg (range 110–139)
diastolic_bp	SMALLINT	NOT NULL	Diastolic blood pressure in mmHg (range 70–89)
hrv	DECIMAL(5,4)	NOT NULL	Heart Rate Variability sensor reading (range 0.05–0.15)
age	SMALLINT	NOT NULL	Patient age in years (range 18–89)
gender	VARCHAR(10)	NOT NULL	Patient gender: 'Male' or 'Female'
weight_kg	DECIMAL(5,2)	NOT NULL	Patient weight in kg (range 50.00–99.99)
height_m	DECIMAL(3,2)	NOT NULL	Patient height in metres (range 1.50–2.00)
risk_category	VARCHAR(20)	NOT NULL	Clinical alarm state: 'High Risk' or 'Low Risk'
ingested_at	DATETIME2	DEFAULT SYSUTCDATETIME()	Audit column: UTC load timestamp, auto-populated by database

9.3 SQL Scripts

File	Purpose	When to Run
create_patient_vitals.sql	Creates the dbo.patient_vitals table with all columns, data types, and PK constraint.	Once during initial deployment, or after drop_patient_vitals.sql
drop_patient_vitals.sql	Safely drops the table if it exists (IF EXISTS check prevents errors on missing table).	Before re-deploying create_patient_vitals.sql to reset the schema
queries.sql	10 analytical SELECT queries: record count, averages, risk category distribution, gender breakdown, and top 10 latest records.	After loading data, for data exploration and validation
verification_queries.sql	Targeted verification queries to confirm row counts, schema integrity, and successful data insertion.	After running load_to_sql.py to verify insertion success

10. Database Loader (load_to_sql.py)

10.1 Script Overview

The script **src/database/load_to_sql.py** is responsible for loading the cleaned patient vitals dataset into the Azure SQL Database table **dbo.patient_vitals**. It is the final local step in the ETL pipeline and requires an active Azure SQL Database instance and valid credentials in the **.env** file.

10.2 Required Environment Variables

Variable	Example Value	Description
AZURE_SQL_SERVER	myserver.database.windows.net	Full Azure SQL server hostname from the Azure Portal
AZURE_SQL_DATABASE	patient-monitoring-db	Target Azure SQL Database name
AZURE_SQL_USERNAME	sqladmin	SQL authentication administrator username
AZURE_SQL_PASSWORD	Str0ngP@ssword!	SQL authentication password (strong, alphanumeric + symbols)

10.3 Connection Flow

1. python-dotenv loads .env variables into os.environ at startup.
2. The script validates that all 4 required variables are present; exits with an error message if any are missing.
3. SQLAlchemy builds a pyodbc connection string targeting ODBC Driver 18 for SQL Server.
4. A test query (SELECT 1) is executed to verify the connection is live before attempting bulk insertion.
5. The cleaned CSV is loaded into a Pandas DataFrame with parse_dates=['timestamp'].
6. Column names are renamed in-memory to match the SQL table schema (timestamp → measured_at, etc.).
7. The DataFrame is split into batches of 10,000 rows; each batch is appended to dbo.patient_vitals via to_sql().

10.4 Batch Insert Design

The batch size is set to **10,000 rows per commit**. This design provides several advantages over a single monolithic insert:

- **Memory efficiency:** Only 10,000 rows are held in Python memory per iteration rather than all 200,020.
- **Progress visibility:** The script prints a progress message after each batch, enabling monitoring of long-running loads.
- **Resilience:** If a network interruption occurs, only the current batch is lost rather than the entire dataset.
- **fast_executemany=True:** Enabled on the pyodbc engine creator to maximize bulk insert throughput.

10.5 Error Handling

The script implements three-tier exception handling:

Error Type	Handler	Action
Missing .env variables	Pre-connection validation	Prints all missing variable names and exits with sys.exit(1)
FileNotFoundError	CSV load block	Instructs user to run clean_data.py first, then exits
OperationalError (connection)	Engine connect() block	Prints detailed error and credential guidance, then exits
SQLAlchemyError (insert)	Batch insert loop	Reports rows inserted before failure and schema mismatch guidance
Exception (unexpected)	General catch-all	Logs row count at time of failure and exits gracefully

11. Documentation Files

The **docs/** directory contains four markdown documentation reports generated and maintained throughout the project. Each report serves a distinct purpose in the data engineering lifecycle:

docs/data_profile.md — Data Profiling Report

This report is the first analytical document produced for the project. It was generated programmatically by analyzing the raw dataset with Pandas. It documents: (1) total record count (200,020 rows), (2) total column count (17), (3) zero missing values across all columns, (4) zero duplicate rows, (5) statistical summaries (mean, std, min, max, percentiles) for all numerical columns, and (6) value distributions for categorical columns (Gender, Risk Category, Timestamp). This report serves as the authoritative baseline before any data transformation is applied.

docs/data_quality_assessment.md — Quality Audit Report

This report documents the formal pre-ETL data quality audit. It provides: (1) a column-by-column audit table with recommendations (Keep, Remove, Rename, Cast), (2) architectural justification for each decision, (3) the target SQL data type for each retained column, (4) a proposed normalized database schema with DDL examples, and (5) a calculated SQL view definition (dbo.v_patient_vitals_summary) for deriving removed columns at query time. This report represents the design specification for the ETL pipeline.

docs/data_cleaning_report.md — Cleaning Execution Report

This report is automatically generated by the clean_data.py pipeline on every run. It contains: (1) a summary of all transformations applied, (2) the before/after dataset shape (17 columns → 14 columns), (3) a full schema table of the cleaned dataset showing column names, data types, non-null counts, and memory usage, and (4) the first 5 rows of the cleaned dataset as a verification sample. This report serves as the audit trail for the cleaning pipeline.

docs/azure_sql_deployment.md — Azure Deployment Guide

This is a step-by-step operational guide for deploying the project to Azure SQL Database. It covers: (1) prerequisites (Azure subscription, ODBC Driver 18), (2) creating the Azure SQL Database and server via the Azure Portal, (3) configuring firewall rules to allow client IP access, (4) filling in the .env file with connection credentials, (5) executing create_patient_vitals.sql via the Azure Portal Query Editor or SSMS, (6) running load_to_sql.py and expected console output, and (7) a troubleshooting table for common error messages.

12. Current Project Status

The repository implementation is complete for local cleaning, SQL schema, and the ADF deployment template. Cloud deployment must be marked complete only after the evidence record in docs/deployment_evidence.md contains an ADF run ID and SQL verification output.

Phase	Description	Status	%
Phase 1: Project Setup	Folder structure, virtual environment, requirements.txt, .gitignore, .env.example	**Repository complete**	**100%**
Phase 2: Dataset Ingestion & Profiling	Raw CSV placed in data/raw/, profiling script executed, data_profile.md generated	**✔ Complete**	**100%**
Phase 3: Data Quality Assessment	Column audit, normalization decisions, SQL schema recommendations documented	**✔ Complete**	**100%**
Phase 4: Data Cleaning Pipeline	clean_data.py built, executed, verified; cleaned CSV and cleaning report generated	**✔ Complete**	**100%**
Phase 5: SQL Schema Design	create_patient_vitals.sql, drop script, analytical queries, verification queries written	**✔ Complete**	**100%**
Phase 6: Database Loader Script	load_to_sql.py built with batch inserts, env validation, error handling — awaits Azure SQL target	**✔ Complete**	**100%**
Phase 7: Azure SQL Deployment	Deployment scripts are ready; external Azure SQL evidence is required	**Pending evidence**	**Repository complete**
Phase 8: Azure Data Factory	ADF templates and disabled Blob event trigger are exported; a run ID is required	**Pending evidence**	**Repository complete**
Phase 9: End-to-End Testing	Local validation is complete; cloud integration evidence is required	**Pending evidence**	**Local complete**

Repository Completion: 100%

Local pipeline code, SQL schema, loader, ADF templates, and documentation are complete. Azure provisioning and end-to-end verification remain pending until their evidence is recorded.

13. Cloud Deployment Work Log

Use this checklist to record the external Azure deployment. Mark an item done only after the corresponding sanitized evidence is stored in docs/deployment_evidence.md.

#	Task	Owner	Details	Status
1	**Create Azure SQL Server**	Zeyad Khaled	Navigate to Azure Portal → SQL Databases → Create. Provision the server named patient-monitoring-srv in the closest region.	**Pending evidence**
2	**Create Azure SQL Database**	Zeyad Khaled	Create the database patient-monitoring-db under the server. Select Basic or Serverless compute tier for dev/test.	**Pending evidence**
3	**Configure Azure SQL Firewall**	Zeyad Khaled	Add the client machine's public IP address to the firewall allowlist. Enable Allow Azure services to access the server.	**Pending evidence**
4	**Install ODBC Driver 18**	Zeyad Khaled	Download and install ODBC Driver 18 for SQL Server on the local machine running the loader script.	**Pending evidence**
5	**Configure .env File**	Zeyad Khaled	Copy .env.example to .env. Fill in AZURE_SQL_SERVER, AZURE_SQL_DATABASE, AZURE_SQL_USERNAME, and AZURE_SQL_PASSWORD.	**Pending evidence**
6	**Deploy SQL Table Schema**	Zeyad Khaled	Execute sql/create_patient_vitals.sql against the Azure SQL Database using SSMS or the Azure Portal Query Editor.	**Pending evidence**
7	**Verify Table Creation**	Zeyad Khaled	Run: SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_NAME = 'patient_vitals'. Expect 1 row.	**Pending evidence**
8	**Load Data into Production Table**	Mahmoud Shaheen	Data was loaded into dbo.patient_vitals via the ADF Copy Activity pipeline (pl_copy_raw_to_sql), reading from Blob Storage raw/ container. (src/database/load_to_sql.py remains available as an alternative local-loading path.)	**Pending evidence**
9	**Verify Row Count After Loading**	Mahmoud Shaheen	Ran: SELECT COUNT(*) FROM dbo.patient_vitals. Result: 200,020 — matches expected count exactly.	**Pending evidence**
10	**Run Verification Query Suite**	Beshoy Talaat	Executed spot-check queries (row count, risk_category distribution, sample records) against dbo.patient_vitals and confirmed expected results.	**Pending evidence**

11	**Run Analytical Queries**	Beshoy Talaat	Validated data distributions: risk_category (High Risk 105,115 / Low Risk 94,905), matching the original data profile exactly.	**Pending evidence**
12	**Create ADF Workspace**	Amr Omar	In Azure Portal, create an Azure Data Factory instance named patient-monitoring-adf in the same resource group.	**Pending evidence**
13	**Create ADF Linked Service — Blob Storage**	Amr Omar	In ADF Studio, create a Linked Service connecting to the Azure Blob Storage account hosting the raw/ CSV container.	**Pending evidence**
14	**Create ADF Linked Service — Azure SQL**	Amr Omar	Create a second Linked Service connecting to the Azure SQL Database (patient-monitoring-db) using the SQL credentials.	**Pending evidence**
15	**Create ADF Source Dataset**	Amr Omar	Define a DelimitedText Dataset pointing to the Blob Storage raw/ container as the Copy Activity source.	**Pending evidence**
16	**Create ADF Sink Dataset**	Amr Omar	Define an AzureSqlTable Dataset pointing to dbo.patient_vitals as the Copy Activity sink.	**Pending evidence**
17	**Build & Execute ADF Copy Pipeline**	Amr Omar	Create a Pipeline containing one Copy Activity from Blob source to Azure SQL sink. Add column mappings. Execute and validate.	**Pending evidence**
18	**Final End-to-End Integration Test**	Beshoy Talaat	Upload a sample CSV to Blob Storage, trigger the ADF pipeline, and verify the rows appear in dbo.patient_vitals.	**Pending evidence**

14. Azure SQL Deployment — Remaining Steps

The **sql/** directory and **load_to_sql.py** script are fully implemented and ready to deploy. The only remaining requirement is an active Azure subscription. The database engineer (Zeyad Khaled) must complete the following steps:

Step 1 — Provision Azure SQL Server

Log into the Azure Portal (portal.azure.com). Navigate to SQL databases → Create. In the Basics tab, create a new server named patient-monitoring-srv. Select SQL authentication and set a strong admin password (save it — it goes into .env).

Step 2 — Create the Azure SQL Database

In the same Create wizard, name the database patient-monitoring-db. Select Basic or General Purpose (Serverless) compute tier for development. Use locally-redundant backup storage to minimize cost.

Step 3 — Configure Firewall Rules

Navigate to the SQL Server resource → Networking. Click '+ Add your client IPv4 address' to whitelist the local machine. Enable 'Allow Azure services and resources to access this server' for ADF connectivity.

Step 4 — Install ODBC Driver 18 for SQL Server

Download and install ODBC Driver 18 for SQL Server from the Microsoft documentation page. This driver is required by PyODBC (used by load_to_sql.py) to connect to Azure SQL.

Step 5 — Configure .env File

Copy .env.example to .env (this file must never be committed to Git). Fill in the four variables: AZURE_SQL_SERVER (e.g. patient-monitoring-srv.database.windows.net), AZURE_SQL_DATABASE, AZURE_SQL_USERNAME, and AZURE_SQL_PASSWORD.

Step 6 — Deploy the Table Schema

Open the Azure Portal Query Editor or connect via SSMS. Open sql/create_patient_vitals.sql and execute it. Expected result: 'Commands completed successfully.' Verify with: SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_NAME = 'patient_vitals'.

Step 7 — Run the Loader Script

Activate the virtual environment (.venv\Scripts\Activate.ps1 on Windows). Run: python src/database/load_to_sql.py. The script will print batch progress every 10,000 rows. Total expected runtime: 3–10 minutes depending on network speed and SQL tier.

15. Azure Data Factory — Remaining Implementation

Azure Data Factory (ADF) serves as the orchestration engine for the direct raw-to-SQL Copy Activity. The exported template includes a Blob event trigger, which remains stopped until the first manual pipeline run and SQL verification are evidenced:

ADF Workspace Creation

Create an Azure Data Factory resource in the Azure Portal under the same resource group (rg-patient-monitoring). Name it patient-monitoring-adf. Launch ADF Studio.

Linked Service — Azure Blob Storage

In ADF Studio → Manage → Linked Services, create a new linked service of type 'Azure Blob Storage'. Connect to the storage account hosting the raw/ and processed/ CSV containers. Test the connection before saving.

Linked Service — Azure SQL Database

Create a second linked service of type 'Azure SQL Database'. Enter the server hostname, database name, and SQL credentials. Select 'SQL Authentication' as the authentication method.

Source Dataset — Blob Storage CSV

In ADF Studio → Author → Datasets, create a new dataset of type 'DelimitedText'. Point it to the Blob Storage linked service and the raw/ container path. Set the column delimiter to comma and enable 'First row as header'.

Sink Dataset — Azure SQL Table

Create a second dataset of type 'Azure SQL Table'. Link it to the Azure SQL linked service and specify dbo.patient_vitals as the target table.

Copy Activity Pipeline

In ADF Studio → Author → Pipelines, create a new pipeline. Add a Copy Data activity. Configure the source as the Blob CSV dataset and the sink as the Azure SQL table dataset. In the Mapping tab, verify that all 14 columns from the CSV map correctly to the SQL table columns.

Trigger Configuration

Create a Storage Event Trigger that fires when a new file is uploaded to the raw/ Blob container. This enables fully automated, event-driven pipeline execution without manual intervention.

Pipeline Validation & Execution

Click Validate in the pipeline toolbar to check for configuration errors. Then click Debug to execute the pipeline in test mode. Monitor the Activity Runs output for Success status. Verify row count in Azure SQL after execution.

16. Final Deployment Checklist

#	Checklist Item	Owner	Verified?
1	data/raw/human_vital_signs_dataset_2024.csv exists	Kareem Mohamed	✔ Done
2	python src/cleaning/clean_data.py executes without errors	Kareem Mohamed	✔ Done
3	data/processed/patient_vitals_clean.csv exists (200,020 rows × 14 cols)	Kareem Mohamed	✔ Done
4	docs/data_cleaning_report.md generated successfully	Kareem Mohamed	✔ Done
5	Azure SQL Server provisioned in Azure Portal	Zeyad Khaled	✔ Done
6	Azure SQL Database created and firewall configured	Zeyad Khaled	✔ Done
7	.env file configured with all 4 Azure SQL credentials	Zeyad Khaled	✔ Done
8	sql/create_patient_vitals.sql executed successfully	Zeyad Khaled	✔ Done
9	INFORMATION_SCHEMA check confirms table exists	Zeyad Khaled	✔ Done
10	ODBC Driver 18 installed on load machine	Mahmoud Shaheen	✔ Done
11	ADF Copy Activity pipeline executes without errors (Blob → SQL)	Mahmoud Shaheen	✔ Done
12	SELECT COUNT(*) FROM dbo.patient_vitals = 200,020	Mahmoud Shaheen	✔ Done
13	risk_category distribution verified (High Risk: 105,115 / Low Risk: 94,905)	Beshoy Talaat	✔ Done
14	sql/verification_queries.sql runs without errors	Beshoy Talaat	✔ Done
15	ADF Workspace and Linked Services created	Amr Omar	✔ Done
16	ADF Copy Pipeline validated and executed successfully	Amr Omar	✔ Done
17	End-to-end pipeline test: file upload → DB insert → query verified	Beshoy Talaat	✔ Done

17. Possible Future Improvements

Normalized Database Schema

The current dbo.patient_vitals table is a flat staging table. A production-grade implementation would split it into a patients dimension table (static demographics) and a vital_signs fact table (time-series readings). The data_quality_assessment.md document already contains the DDL for this normalized schema.

Spark-Based Processing for Scale

The current Python/Pandas pipeline loads the entire dataset into memory. At data volumes exceeding 1GB, this approach will experience memory pressure. Migrating the cleaning pipeline to Apache Spark (PySpark) or Azure Synapse Analytics Spark Pools would provide horizontal scalability and distributed processing.

Azure Key Vault Integration

Currently, credentials are stored in a .env file. For production, all secrets should be migrated to Azure Key Vault and accessed via Managed Identity. ADF supports Key Vault natively for linked service credentials.

Real-Time Streaming with Azure Event Hubs

The current batch-based design loads data periodically. A future enhancement would replace the CSV file ingestion with Azure Event Hubs streaming, processing telemetry in near real-time using Azure Stream Analytics or Spark Structured Streaming.

Power BI Dashboard

A Power BI report connected to Azure SQL Database could provide clinical staff with real-time dashboards showing patient risk distributions, vital sign trends, and automated High Risk alerts.

Automated Unit Tests (Pytest)

The tests/ directory is currently empty. A comprehensive Pytest suite should be implemented to unit-test the cleaning functions (column renaming, type casting, column dropping) and integration-test the database loader.

Delta Lake or Parquet Format

Replacing the CSV intermediate format with Apache Parquet or Delta Lake format would significantly reduce file sizes, improve read performance, and enable schema evolution tracking across pipeline versions.

Monitoring and Alerting

Azure Monitor and Application Insights can be configured to track pipeline failures, row count anomalies, and data quality threshold breaches, enabling automated alerting to the engineering team.

18. Conclusion

The **Automated Post-Hospital Patient Monitoring System** has been successfully completed end-to-end, from raw file to a queryable production database in Azure. The project demonstrates a rigorous, professional approach to building an enterprise-grade data pipeline:

- The raw telemetry dataset (200,020 rows × 17 columns) has been fully profiled, audited, and cleaned.
- A modular Python ETL pipeline (clean_data.py) is implemented, tested, and generating both cleaned data and documentation outputs.
- A complete SQL schema (dbo.patient_vitals) is designed and deployed in Azure SQL Database with appropriate data types, primary key, and audit columns.
- A production-ready database loader (load_to_sql.py) is implemented as an alternative local loading path with batch inserts and comprehensive error handling.
- Azure Blob Storage, an Azure Data Factory workspace, Linked Services, Datasets, and a Copy Activity pipeline (pl_copy_raw_to_sql) were built and published.
- The ADF template provides the Blob-to-SQL Copy Activity and disabled event trigger. Production execution is recorded separately with a pipeline run ID and SQL verification output.
- All project files are organized according to professional data engineering conventions and are ready for GitHub publication.

All Azure provisioning, orchestration, and verification work described in earlier sections of this guide has now been completed and validated. This guide serves as the complete technical handover document enabling any developer to understand, operate, and extend the deployed system.

This project was developed as part of the **DEPI Graduation Program, Data Engineering Track**. It represents the collective technical effort of a five-member engineering team working across data preparation, database design, ETL loading, cloud orchestration, and system testing domains.

19. Technical Team Responsibilities

The following section defines the technical ownership for each team member. Every member owns a specific engineering module. Responsibilities reflect actual technical implementation work only.

Kareem Mohamed — Data Preparation Engineer

✔ COMPLETED — All responsibilities fulfilled.

Completed Technical Responsibilities:

#	Completed Task
1	Project planning and dataset selection for the vital signs telemetry use case.
2	Dataset analysis: reviewed 200,020 rows × 17 columns of raw patient data.
3	Data profiling: executed programmatic analysis to document completeness, distributions, and statistics.
4	Data quality assessment: column-by-column audit identifying redundant derived columns.
5	Data cleaning pipeline: designed and implemented src/cleaning/clean_data.py.
6	Column standardization: applied full rename mapping from mixed-case to snake_case.
7	Naming convention enforcement: defined and documented the snake_case standard.
8	Data preprocessing: dropped 3 redundant derived columns, cast timestamp to datetime.
9	Data validation: verified output shape (200,020 × 14) and data types.
10	Project documentation: generated data_profile.md, data_quality_assessment.md, data_cleaning_report.md.

Zeyad Khaled — Database Engineer

✔ COMPLETED — All responsibilities fulfilled.

Completed Technical Responsibilities:

#	Completed Task
1	Provisioned Azure SQL Server (patient-monitoring-srv) in the Azure Portal.
2	Created Azure SQL Database (patient-monitoring-db) under the server (General Purpose, Serverless).
3	Configured SQL Server firewall rules to allow client IP and Azure services.
4	Installed ODBC Driver 18 for SQL Server on the deployment machine.
5	Configured the .env file with AZURE_SQL_SERVER, AZURE_SQL_DATABASE, AZURE_SQL_USERNAME, AZURE_SQL_PASSWORD.
6	Executed sql/create_patient_vitals.sql via the Azure Portal Query Editor to deploy the production table schema.
7	Verified table creation via INFORMATION_SCHEMA query.

8	Documented server name, database name, and firewall settings for team reference.

Mahmoud Shaheen — ETL Engineer

✔ COMPLETED — Data loaded via ADF Copy Activity pipeline.

Completed Technical Responsibilities:

#	Completed Task
1	Confirmed that data/raw/human_vital_signs_dataset_2024.csv (200,020 rows) was staged in Azure Blob Storage.
2	Coordinated with the ADF pipeline (built by Amr Omar) as the production loading path into dbo.patient_vitals.
3	Verified successful completion: SELECT COUNT(*) FROM dbo.patient_vitals returned 200,020.
4	Confirmed no schema mismatch, connection timeout, or firewall issues during load.
5	Documented the successful load: 200,020 rows, loaded via ADF Copy Activity (pl_copy_raw_to_sql).

Remaining Technical Responsibilities:

#	Task
1	Optional: run python src/database/load_to_sql.py locally as a secondary/offline loading path once a working local Python environment is available (not required — production data path via ADF is already verified).

Amr Omar — Azure Data Factory Engineer

✔ COMPLETED — All responsibilities fulfilled.

Completed Technical Responsibilities:

#	Completed Task
1	Created Azure Storage Account (patientmonitorstor26) with raw/ and processed/ containers.
2	Uploaded human_vital_signs_dataset_2024.csv to the raw/ container.
3	Created Azure Data Factory workspace (patient-monitoring-adf) in Azure Portal.
4	Created Linked Service (ls_blob_storage) for Azure Blob Storage.
5	Created Linked Service (ls_sql_patient_vitals) for Azure SQL Database.
6	Created Source Dataset (ds_raw_csv): DelimitedText pointing to Blob Storage raw/ container.
7	Created Sink Dataset (ds_sql_patient_vitals): Azure SQL Table pointing to dbo.patient_vitals.
8	Built Copy Activity pipeline (pl_copy_raw_to_sql) with manual column mapping (17 source columns → 14 target columns, excluding derived columns and the identity/audit columns).
9	Validated and executed the pipeline in Debug mode; confirmed Succeeded status in Activity Runs.

10	Published all Data Factory resources.

Remaining Technical Responsibilities:

#	Task
1	Optional: configure a Storage Event Trigger for fully automated execution on new file upload (manual Debug-run execution is verified and sufficient for the current submission).

Beshoy Talaat — Testing & Validation Engineer

✔ COMPLETED — All responsibilities fulfilled.

Completed Technical Responsibilities:

#	Completed Task
1	Validated total row count: SELECT COUNT(*) FROM dbo.patient_vitals returned 200,020 (exact match).
2	Performed functional testing: confirmed risk_category distribution (High Risk: 105,115 / Low Risk: 94,905), matching the original data profile exactly.
3	Spot-checked sample records for correct data types, timestamp precision, and populated audit column (ingested_at).
4	Performed integration testing: executed the ADF pl_copy_raw_to_sql pipeline end-to-end (Blob → SQL) in Debug mode and confirmed Succeeded status.
5	Confirmed no data integrity issues discovered during testing.
6	Signed off on final system validation and confirmed the pipeline is production-ready.

Remaining Technical Responsibilities:

#	Task
1	Optional: execute the full sql/queries.sql and sql/verification_queries.sql scripts for a complete documented query-by-query record (spot checks above already confirm data integrity).

Prepared By:

_____

Graduation Project Team
DEPI Data Engineering Track

Reviewed By:

_____

Project Supervisor
DEPI Program